

Tài liệu chi tiết cho dự án RAE XLA và JAX

Nhánh TorchXLA/TPU và lớp tương thích JAX/NNX của dự án Representation Autoencoders

Ngày 8 tháng 4 năm 2026

Mục lục

1	Tổng quan dự án	3
1.1	Mục tiêu	3
1.2	Hai giai đoạn chính	3
1.2.1	Stage 1	3
1.2.2	Stage 2	3
1.3	Phạm vi của branch hiện tại	3
1.4	Giới hạn hiện tại	3
1.5	Dòng dữ liệu cấp cao	4
1.6	Ý tưởng của đường JAX	4
1.7	Tài liệu trong repo	4
2	Kiến trúc hệ thống	4
2.1	Bản đồ codebase	4
2.2	Stage 1: Representation Autoencoder và StabilityVAE	5
2.3	Stage 2: SiT trong không gian latent	5
2.4	Transport objective	6
2.5	Lớp adapter JAX	6
2.6	Hai mô hình thực thi	7
2.6.1	TorchXLA	7
2.6.2	JAX / NNX	7
2.7	Checkpoint và tương thích trọng số	7
3	Workflow thực chiến	8
3.1	Cài đặt môi trường	8
3.1.1	Đường XLA	8
3.1.2	Phần bổ sung cho JAX / NNX	8
3.2	Chuẩn bị model và dữ liệu	8
3.2.1	Model	8
3.2.2	Dữ liệu	9
3.3	Kiểm tra Stage 1 bằng reconstruction	9
3.3.1	Đường XLA / PyTorch	9
3.3.2	Đường JAX	9
3.3.3	Tính latent stat cho Stage 1 trên đường JAX	9
3.3.4	Reconstruction cả thư mục trên đường JAX	9
3.4	Huấn luyện Stage 2 trên TPU bằng TorchXLA	10

3.5	Huấn luyện Stage 2 bằng JAX / NNX	10
3.6	Wandb logging	11
3.7	FID trong train loop	11
3.8	Sampling Stage 2	12
3.8.1	JAX một lệnh	12
3.8.2	JAX distributed	12
3.9	Đẩy artifact lên Hugging Face	12
3.9.1	Lệnh độc lập	12
3.9.2	Tích hợp ngay trong train	12
3.10	Notebook Kaggle cho CelebA	12
4	Cấu hình và ánh xạ sang JAX	14
4.1	Các block config chính	14
4.2	Ma trận script và block config	14
4.3	Block stage_1	14
4.4	Block stage_2	15
4.5	Block guidance	15
4.6	Block training	16
4.7	Block eval	16
4.8	Override từ CLI	17
5	Vận hành, artifact và FID	17
5.1	Sampling Stage 2 trên đường JAX	17
5.1.1	Sampling nhanh	17
5.1.2	Sampling phân tán	17
5.2	Build reference statistics cho FID	18
5.3	Artifact của Stage 2 training	18
5.3.1	Đường XLA	18
5.3.2	Đường JAX	18
5.4	Đẩy model lên Hugging Face	18
5.5	Các lưu ý vận hành quan trọng	19
5.6	Checklist thực tế trước khi chạy dài	19

1. Tổng quan dự án

1.1. Mục tiêu

Dự án này triển khai pipeline tạo ảnh hai giai đoạn dựa trên *Representation Autoencoders* (RAE). Ở thời điểm hiện tại, repo có hai đường chạy chính:

- `src/`: nhánh TorchXLA, tối ưu cho TPU với train và sampling Stage 2, reconstruction Stage 1, và FID phía host;
- `src_jax/`: lớp tương thích JAX/NNX, giữ nguyên schema YAML của repo nhưng ánh xạ sang backend NNX để tránh nhân đôi toàn bộ codebase.

1.2. Hai giai đoạn chính

1.2.1. Stage 1

Stage 1 dùng một encoder biểu diễn đã được pretrain và đóng băng, kết hợp với một decoder ViT để tái tạo ảnh. Đầu ra của Stage 1 là latent giàu ngữ nghĩa hơn latent VAE thông thường.

1.2.2. Stage 2

Stage 2 học mô hình diffusion transformer trong không gian latent của Stage 1. Thay vì học trực tiếp trên pixel, mô hình học quy luật biến đổi của latent, sau đó giải mã latent sinh được thành ảnh RGB qua decoder của RAE.

1.3. Phạm vi của branch hiện tại

Nhánh hiện tại không chỉ có đường TorchXLA mà còn có adapter JAX mỏng. Phần được hỗ trợ thực dụng nhất là:

- huấn luyện Stage 2 bằng `src/train.py` trên TPU;
- huấn luyện Stage 2 bằng `src_jax/train.py` trên JAX/NNX;
- sampling Stage 2 bằng cả `src/` và `src_jax/`;
- reconstruction Stage 1 để kiểm tra encoder-decoder trên cả hai đường;
- logging qua wandb;
- upload artifact hoặc workdir lên Hugging Face;
- FID offline, và online FID trong train loop ở đường XLA.

1.4. Giới hạn hiện tại

- `src/` vẫn không có endpoint riêng cho huấn luyện Stage 1 trên XLA;
- `src_jax/` cố ý chưa port toàn bộ vòng huấn luyện Stage 1 kiểu GAN + LPIPS + discriminator, vì phần đó sẽ làm codebase lớn và khó bảo trì hơn đáng kể.

1.5. Dòng dữ liệu cấp cao

Bước	Mô tả
1	Ảnh đầu vào được đưa qua encoder của Stage 1
2	Encoder sinh latent tensor có kích thước như <code>misc.latent_size</code>
3	Stage 2 học transport objective trên latent đó
4	Khi sampling, latent được sinh bởi Stage 2
5	Decoder của Stage 1 giải mã latent thành ảnh RGB

1.6. Ý tưởng của đường JAX

Đường JAX không tạo thêm một schema config mới. Thay vào đó:

- vẫn đọc các block `stage_1`, `stage_2`, `transport`, `sampler`, `guidance`, `misc`, `training`, `eval`;
- dùng `src_jax/config_adapter.py` để chuyển YAML hiện có sang config mà backend NNX hiểu được;
- bootstrap backend `diffuse_nnx` theo commit cố định để branch gọn hơn, không phải chép toàn bộ framework vào repo.

1.7. Tài liệu trong repo

Ngoài PDF này, repo còn có bộ tài liệu markdown trong `docs/`. Phần markdown thiên về runbook ngắn và tra cứu nhanh; PDF này gom lại kiến trúc, workflow, cấu hình và vận hành cho cả đường XLA lẫn JAX.

2. Kiến trúc hệ thống

2.1. Bản đồ codebase

```

configs/
  stage1/
  stage2/
src/
  stage1/
  stage2/
  utils/
  train.py
  sample.py
  sample_ddp.py
  stage1_sample.py
  stage1_sample_ddp.py
  build_fid_stats.py
  evaluate_fid.py
src_jax/
  vendor.py
  config_adapter.py
  stage2_runtime.py
  stage1_runtime.py
  build_stage1_stats.py
  export_celebahq_hf.py
  export_celebahq_tfds.py

```

```

reconstruct_folder.py
hf_utils.py
train.py
sample.py
sample_ddp.py
stage1_sample.py
push_hf.py
vae-jax-celeba-kaggle-tpuv5e8-sitb.ipynb
vae-jax-celeba-kaggle-tpuv5e8-sitb-resume.ipynb

```

2.2. Stage 1: Representation Autoencoder và StabilityVAE

Thành phần gốc của Stage 1 nằm trong `src/stage1/rae.py`. Lớp RAE kết hợp:

- encoder ở `src/stage1/encoders/`,
- decoder ở `src/stage1/decoders/`,
- latent normalization qua `normalization_stat_path`,
- latent noising qua `noise_tau`.

Hành vi chính:

- `encode(x)` resize đầu vào về đúng độ phân giải của encoder, chuẩn hóa theo image processor, chạy encoder, reshape latent về dạng 2D nếu cần, rồi áp normalization;
- `decode(z)` đảo normalization, đổi latent về dạng sequence nếu cần, rồi giải mã thành ảnh RGB bằng decoder ViT.

Ở nhánh này, `src/stage1/stability_vae.py` còn cung cấp target `stage1.StabilityVAE` như một alias repo-facing cho encoder `StabilityVAE` native của backend NNX. Đường này:

- không cần `pretrained_decoder_path` của RAE;
- mặc định tự materialize checkpoint `vae_trial1.pkl` cục bộ từ `stabilityai/sd-vae-ft-mse`, trừ khi `stage_1.params.pretrained_path` đã trỏ tới pickle sẵn có;
- không bắt buộc phải bootstrap latent stat trước khi chạy CelebA;
- mặc định suy ra latent shape `[4, 32, 32]` ở độ phân giải 256x256.

2.3. Stage 2: SiT trong không gian latent

Surface Stage 2 dùng chung của branch này nằm ở `src/stage2/models/SiT.py`. File này giờ có hai alias repo-facing:

- `SiTDH`: backbone DH/two-tower dựa trên `DiTwDDTHead`;
- `SiT`: backbone single-tower dựa trên `LightningDiT`.

Nhờ đó nhánh hiện tại vẫn giữ được đường `SiTDH` ở mức code/config khi cần, đồng thời public notebook flow của CelebA tập trung vào `StabilityVAE + SiT-B`. Objective trên đường JAX vẫn đi qua interface `sit` cho cả hai dạng backbone.

Các đặc điểm quan trọng:

- đầu vào là latent thay vì pixel;
- embedding thời gian bằng Gaussian Fourier features;
- class conditioning bằng LabelEmbedder;
- tùy chọn ROPE, RMSNorm, SwiGLU, qk-norm;
- hỗ trợ classifier-free guidance và autoguidance khi sampling.

Đường JAX ánh xạ `stage_2.target` sang backbone tương ứng của backend NNX:

- SiTDH $\rightarrow$ `lightning_ddt`,
- SiT $\rightarrow$ `lightning_dit`,
- LightningDiT $\rightarrow$ `lightning_dit`,
- các target DDT legacy $\rightarrow$ `lightning_ddt`,
- các target kiểu DiT khác $\rightarrow$ `dit`.

2.4. Transport objective

Phần này nằm trong `src/stage2/transport/transport.py`. Đây là lớp trung gian giữa latent diffusion model và train loop.

Nhiệm vụ của `transport`:

- lấy mẫu thời gian t ;
- xây dựng đường nối giữa noise và dữ liệu thật;
- tính target phù hợp với loại output của model;
- trả về drift function để phục vụ ODE hoặc SDE sampler.

Ở đường JAX, adapter không giữ lại implementation `transport` gốc này. Nó ánh xạ các ý nghĩa tương đương sang interface của backend NNX, chủ yếu là SiT với sampling kiểu Euler.

2.5. Lớp adapter JAX

File	Vai trò
<code>src_jax/vendor.py</code>	Bootstrap <code>diffuse_nnx</code> vào ■ <code>/.cache/rae_jax/diffuse_nnx</code> và pin commit
<code>src_jax/config_adapter.py</code>	Chuyển YAML OmegaConf của repo sang config backend NNX, map SiT/SiTDH/StabilityVAE sang backend phù hợp, forward <code>random_flip</code> / <code>prefetch knobs</code> , và giữ interface train <code>sit</code> cho cả flow DH lẫn single-tower VAE
<code>src_jax/stage2_runtime.py</code>	Train, sampling, load checkpoint PyTorch hoặc Orbax, JAX validation-loss integration, FID glue, wandb bridge
<code>src_jax/stage1_runtime.py</code>	Load encoder Stage 1 JAX, reconstruction một ảnh, reconstruction cả thư mục và tích lũy latent stat cho cả RAE lẫn StabilityVAE

File	Vai trò
<code>src_jax/build_stage1_stats.py</code>	Export file <code>stat.pt</code> theo định dạng <code>mean / var</code> để dùng lại cho Stage 1
<code>src_jax/export_celebahq_hf.py</code>	Download <code>eurecom-ds/celeba-hq-256</code> từ Hugging Face rồi export thành cây <code>ImageFolder</code> cho pipeline JAX hiện tại
<code>src_jax/export_celebahq_tfds.py</code>	Export TFDS <code>celeb_a_hq/256</code> thành cây <code>ImageFolder</code> cho pipeline JAX hiện tại và ép <code>protobuf runtime</code> kiểu Python để tránh lỗi <code>import TFDS</code> trên Kaggle
<code>src_jax/build_fid_stats.py</code>	Build <code>fid_ref</code> bằng đúng detector <code>Flax Inception</code> của backend JAX
<code>src_jax/reconstruct_folder.py</code>	Export reconstruction cho cả thư mục ảnh theo pipeline JAX
<code>src_jax/hf_utils.py</code>	Upload file hoặc thư mục lên Hugging Face

2.6. Hai mô hình thực thi

2.6.1. TorchXLA

Các entrypoint distributed trong `src/` dùng `xmp.spawn(...)` và mỗi TPU core chạy một process riêng. Các primitive quan trọng:

- `DistributedSampler`,
- `ParallelLoader`,
- `xm.optimizer_step(...)`,
- `xm.rendezvous(...)` và `xm.mesh_reduce(...)`.

2.6.2. JAX / NNX

Đường `src_jax/` không tự quản lý toàn bộ graph, checkpoint và sampler ở mức thấp. Nó dựa vào backend NNX để:

- tạo model, optimizer, sampler và EMA,
- quản lý checkpoint Orbx,
- chạy sampling và FID trong định dạng tensor NHWC của JAX,
- mở rộng sang nhiều process JAX nếu môi trường đã cấu hình phù hợp.

Repo cũng vá backend này để EMA khởi tạo từ bản copy của model hiện thời thay vì một cây tham số toàn số 0. Nhờ vậy `ema_network_samples` và online FID không còn bắt đầu từ một EMA cold-start sai ngay từ step đầu.

2.7. Checkpoint và tương thích trọng số

Với branch `jax-vae-sit`, checkpoint Stage 2 phải khớp shape của backbone đã chọn:

- nếu `stage_2.ckpt` là file PyTorch `.pt`, adapter sẽ port state dict sang NNX để inference hoặc khởi tạo train nếu checkpoint đó đã tương thích với SiT hoặc SiTDH;

- nếu `stage_2.ckpt` là thư mục checkpoint Orbax, adapter sẽ restore trực tiếp run JAX trước đó với cùng shape backend;
- checkpoint DDT legacy không được tự động convert trên branch này;
- decoder Stage 1 ở đường JAX vẫn dùng được checkpoint decoder PyTorch trên flow RAE; flow StabilityVAE dùng trọng số VAE native của backend.

3. Workflow thực chiến

3.1. Cài đặt môi trường

Repo hiện có hai cụm phụ thuộc chính.

3.1.1. Đường XLA

```
conda create -n rae python=3.10 -y
conda activate rae
pip install uv
uv pip install torch~=2.5.0 torch_xla[tpu]~=2.5.0 torchvision==0.20.1 -f https://
    storage.googleapis.com/libtpu-releases/index.html
uv pip install timm==0.9.16 accelerate==0.23.0 torchdiffeq==0.2.5 wandb scipy torch-
    fidelity
uv pip install "numpy<2" "transformers==4.57.1" einops
```

3.1.2. Phần bổ sung cho JAX / NNX

```
uv pip install "jax[cuda12]==0.5.1" flax==0.10.4 optax==0.2.4 orbax-checkpoint==0.11.16
uv pip install ml-collections clu absl-py etils datasets diffusers huggingface_hub
    tensorflow-datasets
```

Lần chạy JAX đầu tiên sẽ tự bootstrap backend `diffuse_nnx` vào thư mục `./.cache/rae_jax/diffuse_nnx`. Các notebook JAX trên Kaggle không còn lặp lại từng lệnh `uv pip install` này; chúng đặt `UV_PROJECT_ENVIRONMENT=UV_CACHE_DIR=/tmp/uv-cache`, rồi chạy `uv sync -q` theo `pyproject.toml` của repo trước các cell phụ thuộc package. Repo này tự vá backend `diffuse_nnx` để import các model Dinov2 từ subpackage của `transformers`, nên nên dùng `transformers==4.57.1` cho đường này. Patch này cũng chuyển `google-cloud-storage` sang import lười cho các asset encoder còn đi qua GCS, còn đường `public stage1.StabilityVAE` sẽ tự materialize `vae_trial1.pkl` cục bộ từ `stabilityai/sd-vae-ft-mse` thay vì phụ thuộc bucket cũ của backend. Patch bootstrap này cũng sửa khởi tạo EMA của backend để EMA bắt đầu từ bản copy của model hiện thời thay vì cây tham số toàn số 0. Nếu bạn resume một checkpoint Orbax cũ được tạo trước khi có fix này, checkpoint đó vẫn mang EMA đã lưu sẵn trước đó. Adapter cũng tự suy ra `downsample_factor` và `latent_channels` từ `misc.latent_size`, nên bước preview sample và FID của backend vẫn chạy ở latent resolution thay vì lỗi cấp phát noise theo image resolution.

3.2. Chuẩn bị model và dữ liệu

3.2.1. Model

```
hf download nyu-visionx/RAE-collections --local-dir models
```


3.2.2. Dữ liệu

Hầu hết các script dùng dữ liệu dạng ImageFolder. Ví dụ:

```
dataset_root/
  class_a/
    0001.png
  class_b/
    0002.png
```

3.3. Kiểm tra Stage 1 bằng reconstruction

3.3.1. Đường XLA / PyTorch

```
python src/stage1_sample.py \
  --config <config> \
  --image assets/pixabay_cat.png \
  --output recon.png
```

3.3.2. Đường JAX

```
python3 src_jax/stage1_sample.py \
  --config <config> \
  --image assets/pixabay_cat.png \
  --output recon_jax.png
```

Lệnh này phù hợp để xác nhận:

- checkpoint decoder Stage 1 có load được hay không,
- latent shape có đúng hay không,
- preprocessing của encoder có khớp hay không.

3.3.3. Tính latent stat cho Stage 1 trên đường JAX

Với dataset mới như CelebA, nên tạo một file bootstrap identity stat trước (mean = 0, var = 1), rồi chạy:

```
python3 src_jax/build_stage1_stats.py \
  --config <stage1_config> \
  --input <train_imagefolder> \
  --output <stage1_stat.pt> \
  --batch-size 16 \
  --set stage_1.params.normalization_stat_path=<bootstrap_identity_stat.pt>
```

File stage1_stat.pt tạo ra vẫn cùng định dạng với repo gốc, nên dùng lại được cho cả đường src/ lẫn src_jax/. Các lệnh Stage 1 thuần ở JAX có thể chạy với YAML chỉ khai báo stage_1; chúng không bắt buộc có stage_2.target.

3.3.4. Reconstruction cả thư mục trên đường JAX

```
python3 src_jax/reconstruct_folder.py \
  --config <stage1_config> \
  --input <val_imagefolder> \
```

```
--output-dir recon_jax_dir \
--batch-size 8
```

Lệnh này hữu ích khi muốn export reconstruction set trước khi build FID reference hoặc khi cần so sánh decoder Stage 1 giữa các checkpoint.

3.4. Huấn luyện Stage 2 trên TPU bằng TorchXLA

```
python src/train.py \
  --config configs/stage2/training/ImageNet256/SiTdH-XL_DINOv2-B.yaml \
  --data-path <imagenet_train_split> \
  --results-dir results \
  --image-size 256 \
  --precision bf16
```

Trong mỗi optimizer step, train loop thực hiện:

1. center crop và đưa ảnh thành tensor;
2. mã hóa ảnh bằng RAE;
3. tính transport loss trong latent space;
4. step optimizer và scheduler trên TPU;
5. update EMA;
6. chạy logging, checkpointing, preview sample, validation và FID theo cadence.

3.5. Huấn luyện Stage 2 bằng JAX / NNX

```
python3 src_jax/train.py \
  --config configs/stage2/training/ImageNet256/SiTdH-XL_DINOv2-B.yaml \
  --data-path <imagenet_train_root> \
  --results-dir results_jax \
  --precision bf16 \
  --wandb
```

Trên Kaggle TPU, nên bắt đầu với `-set training.num_workers=1`, rồi tăng `-set training.prefetch_factor=<n>` trước khi nhảy thẳng sang một pool worker lớn. Cách này vẫn giữ backend PyTorch loader tương thích với `persistent_workers=True` sau khi JAX đã khởi tạo runtime đa luồng, nhưng cho phép host queue trước nhiều batch hơn để giảm các spike do I/O và decode. Adapter cũng tự tắt backend TensorBoard summary writer trên Kaggle và giữ metric ở stdout cùng wandb.

Điểm quan trọng:

- vẫn dùng cùng file YAML của repo;
- hỗ trợ `-set key=value` để override nhanh từ CLI;
- hỗ trợ `-wandb-run-id <existing_run_id>` để bind một legacy workdir với đúng W&B run cũ đúng một lần;
- nếu gọi resume với `-workdir` nhưng không truyền `-exp-name`, adapter sẽ tự lấy tên run từ `wandb_run.json` khi có, nếu không thì dùng basename của workdir;

- nếu `stage_2.ckpt` là file PyTorch `.pt`, adapter sẽ port trọng số sang NNX để khởi tạo;
- nếu `stage_2.ckpt` là thư mục Orbx, adapter sẽ restore run JAX trước đó.

Nếu bật `training.log_rae_latent_stats=true`, train loop JAX sẽ log `train_rae_latent_rms` và `train_rae_latent_var`. Nếu bật `training.log_activation_stats=true`, backend sẽ trả thêm intermediate feature của SiTDH để log RMS/variance cho output và từng block encoder/decoder, ví dụ `train_sitdh_output_rms`, `train_sitdh_act_enc_00_rms`, và `train_sitdh_act_dec_01_var`. Bù lại, các metric activation này có chi phí throughput và bộ nhớ rõ rệt hơn train loop mặc định. Nếu cần giảm độ trễ host-side, có thể override thêm `training.prefetch_factor` và `eval.prefetch_factor`; các key này được adapter forward thẳng sang DataLoader của đường JAX.

3.6. Wandb logging

Biến môi trường khuyến nghị:

```
export ENTITY=<wandb_entity>
export PROJECT=<wandb_project>
export WANDB_KEY=<wandb_api_key>
```

Đường JAX cũng chấp nhận các tên `WANDB_ENTITY` và `WANDB_API_KEY`. Adapter sẽ bridge các tên legacy ở trên nếu chúng đã được export sẵn. Với `src_jax/train.py`, mỗi workdir mới giờ sẽ ghi `wandb_run.json` để khóa đúng W&B run id của workdir đó. Run mới dùng `resume="never"`; các lần resume sau sẽ đọc lại file này, giữ đúng run id đã bind, rồi tự rewind W&B history về latest checkpoint_* step qua `resume_from`. Nhờ đó phần metric đã log sau checkpoint cuối của session cũ sẽ không bị giữ lại sai lịch sử. Với train resume, runtime giờ sẽ xóa ngay thư mục checkpoint_* vừa restore khỏi ổ đĩa sau khi state đã vào bộ nhớ; trước mỗi lần save checkpoint mới nó cũng xóa các checkpoint_* cũ còn lại để workdir không cần chỗ cho hai Orbx checkpoint cùng lúc. Đường cleanup này giờ cũng chuẩn hóa cả Path lẫn workdir dạng chuỗi mà backend trainer truyền vào, nên resume DH/VAE không còn crash chỉ vì prune checkpoint với `str(workdir)`. Nếu resume một Orbx workdir cũ chưa có `wandb_run.json`, cần truyền `-wandb-run-id <existing_run_id>` một lần để bind đúng historical run trước khi train tiếp. Khi đã có `wandb_run.json`, launch resume không cần lặp lại thủ công `-exp-name`; adapter sẽ tự suy ra tên run từ metadata đó, hoặc từ tên thư mục workdir nếu metadata chưa có. Nếu account/workspace chưa được bật W&B rewind private preview nên `resume_from` bị từ chối, adapter sẽ tự fallback sang `id=<run_id>`, `resume="must"` để launch resume không bị crash chỉ vì thiếu quyền rewind.

3.7. FID trong train loop

Đường XLA hỗ trợ online FID trong `src/train.py`. Ví dụ:

```
eval:
  fid_ref: /path/to/reference_stats.npz
  fid_every: 25000
  fid_num_samples: 4096
  fid_per_proc_batch_size: 4
  fid_batch_size: 128
  fid_device: cpu
  fid_num_threads: 96
```

Đường JAX giờ cũng dùng block `eval` để chạy validation loss trên held-out ImageFolder, log `eval/ema_loss` mặc định và thêm `eval/model_loss` nếu bật `eval_model`. Với online FID, nên xây `fid_ref` bằng

detector backend-native qua:

```
python3 src_jax/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.pkl \
  --batch-size 64 \
  --num-workers 8
```

Trên đường JAX, metric FID-4K (cfg=...) mặc định vẫn chằm EMA. Nếu bật `fid_eval_model`, adapter sẽ log thêm metric riêng FID-4K/model (cfg=...) cho online model để bạn đối chiếu trực tiếp với panel `network_samples`.

3.8. Sampling Stage 2

3.8.1. JAX một lệnh

```
python3 src_jax/sample.py \
  --config configs/stage2/sampling/ImageNet256/SiTDHXL-DINOv2-B_AG.yaml \
  --class-labels 207,360 \
  --output sample_jax.png
```

3.8.2. JAX distributed

```
python3 src_jax/sample_ddp.py \
  --config configs/stage2/sampling/ImageNet256/SiTDHXL-DINOv2-B.yaml \
  --sample-dir samples_jax \
  --num-samples 50000 \
  --label-sampling equal \
  --fid-ref /path/to/reference_stats.npz
```

Các config sampling JAX được commit dưới dạng template. Trước khi chạy sampling, cần tự điền `stage_2.ckpt` và, nếu dùng autoguidance, cả `guidance.guidance_model.ckpt` bằng checkpoint SiTDH tương thích.

3.9. Đẩy artifact lên Hugging Face

3.9.1. Lệnh độc lập

```
python3 src_jax/push_hf.py \
  --path results_jax/<run_name> \
  --repo-id <hf_user_or_org>/<repo_name>
```

3.9.2. Tích hợp ngay trong train

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

3.10. Notebook Kaggle cho CelebA

Nhánh này giữ một public notebook flow cho CelebA:

- **StabilityVAE + SiT-B**: baseline single-tower của nhánh hiện tại.

Giống branch `jax-sit-dh`, public notebook flow bắt đầu từ dataset Kaggle `jessicali9530/celeba-dataset`, center crop rồi resize ảnh về 256x256, sau đó materialize dữ liệu thành cây ImageFolder `celeba256_imgfolder`.

Nếu vẫn muốn đi theo route CelebA-HQ cũ, repo giữ lại `src_jax/export_celebahq_hf.py` và `src_jax/export_celeba_hf.py` nhưng đó không còn là public flow mặc định của branch này.

Nếu muốn dùng baseline VAE được ship sẵn, bắt đầu từ notebook TPU `vaes-jax-celeba-kaggle-tpuv5e8-sitb.ipynb`. Notebook này giữ nguyên flow `uv_sync` và dựng ImageFolder từ CelebA như branch `jax-sit-dh`, nhưng đổi Stage 1 sang `stage1.StabilityVAE` và Stage 2 sang `single-tower stage2.models.SiT.SiT`. Các config được sinh runtime là:

- `configs/stage1/pretrained/CelebA256_StabilityVAE_jax.yaml`;
- `configs/stage2/training/CelebA256_SiT-B_StabilityVAE_jax.yaml`.

Khác với flow RAE:

- không có bước `hf download nyu-visionx/RAE-collections`;
- không cần bootstrap identity stat;
- không bắt buộc phải chạy `src_jax/build_stage1_stats.py` trước khi train Stage 2;
- bật `training.random_flip=true`, `training.log_rae_latent_stats=true`, và `training.log_activation_stats=true` trong config Stage 2 sinh ra;
- cell train cuối truyền `-data-path /kaggle/working/celeba256_imgfolder`, còn `eval.data_path` trong config sinh ra vẫn trỏ tới `split /kaggle/working/celeba256_imgfolder/val`.

Nếu chạy trên Kaggle TPU `v5e-8`, dùng notebook `vaes-jax-celeba-kaggle-tpuv5e8-sitb.ipynb`. Bản này ghi config Stage 2 thành `CelebA256_SiT-B_StabilityVAE_jax_tpuv5e8.yaml`, dùng shape kiểu DiT-B là `hidden_size=768`, `depth=12`, `num_heads=12`, `patch_size=2`, và vẫn giữ objective `sit` / flow matching của repo này. Notebook TPU cũng mặc định cả train lẫn eval ở `num_workers=16` với `prefetch_factor=4`, đồng thời giữ latent RMS/variance, output RMS/variance, và activation RMS/variance bật sẵn.

Nếu đã có run Orbx của flow VAE và cần train tiếp, dùng notebook `vaes-jax-celeba-kaggle-tpuv5e8-sitb-resume.ipynb`. Notebook này bám theo pattern `resume-only` của branch, nhưng tìm `workdir` mới nhất theo pattern `CelebA256_SiT-B_StabilityVAE_jax_tpuv5e8-*` và truyền lại `training.num_workers=16`, `training.prefetch_factor=4`, `eval.num_workers=16`, `eval.prefetch_factor=4`, `training.log_rae_latent_stats=true`, và `training.log_activation_stats=true` khi resume. Nếu `workdir` đích là legacy run được tạo trước khi cơ chế `wandb_run.json` tồn tại, cần thêm `-wandb-run-id <existing_run_id>` một lần để notebook resume đúng chính run W&B cũ đó.

Branch này cũng giữ song song bộ notebook TPU CelebA-HQ: `vaes-jax-celebahq-kaggle-tpuv5e8-sitb.ipynb`, và `vaes-jax-celebahq-kaggle-tpuv5e8-sitb-resume.ipynb`. Chúng dùng cùng pipeline `StabilityVAE + SiT-B`, nhưng export dữ liệu sang `/kaggle/working/celebahq256_imgfolder` và resume theo pattern `CelebAHQ256_SiT-B_StabilityVAE_jax_tpuv5e8-*` thay vì flow CelebA thường. Chúng cũng pin cùng bộ mặc định loader/chẩn đoán Stage 2 như notebook CelebA: `training.num_workers=16`, `training.prefetch_factor=4`, `eval.prefetch_factor=4`, `training.log_rae_latent_stats=true`, và `training.log_activation_stats=true`.

4. Cấu hình và ánh xạ sang JAX

4.1. Các block config chính

Tất cả entrypoint quan trọng đều dựa trên YAML của repo. Các block top-level:

- stage_1,
- stage_2,
- transport,
- sampler,
- guidance,
- misc,
- training,
- eval.

4.2. Ma trận script và block config

Block	src_jax/train.py	src_jax/sample.py	src_jax/sample_eval.py	stage1_sample.py
stage_1	yes	yes	yes	yes
stage_2	yes	yes	yes	no
transport	yes	yes	yes	no
sampler	yes	yes	yes	no
guidance	yes	yes	yes	no
misc	yes	yes	yes	no
training	yes	no	no	no
eval	yes	no	no	no

4.3. Block stage_1

Adapter JAX không tạo schema mới cho Stage 1. Với flow RAE, nó đọc trực tiếp:

- encoder_config_path hoặc encoder_params.dinov2_path,
- pretrained_decoder_path,
- normalization_stat_path,
- encoder_input_size.

Các giá trị này được chuyển sang encoder RAE của backend NNX để phục vụ reconstruction và decode latent khi sampling Stage 2.

Nhánh hiện tại còn có path stage1.StabilityVAE. Với path này, notebook JAX CelebA chỉ cần các trường:

```

stage_1:
  target: stage1.StabilityVAE
  params:
    sample_size: 256
    latent_channels: 4
    downsample_factor: 8
    raw_mean: [0.865, -0.278, 0.216, 0.374]
    raw_std: [4.86, 5.32, 3.94, 3.99]
    final_mean: 0.0
    final_std: 0.5

```

Path này không cần pretrained_decoder_path hay normalization_stat_path; adapter sẽ map nó sang encoder StabilityVAE native của backend và có thể tự suy ra latent shape [4, 32, 32] khi config chưa có stage_2. Nếu đã có checkpoint VAE tương thích, bạn có thể truyền thêm stage_1.params.pretrained_path. Nếu không truyền, lần chạy đầu tiên sẽ tự materialize vae_trial1.pkl cục bộ từ stabilityai/sd-vae-ft-mse.

Nếu cần tính stat riêng cho dataset mới, có thể dùng:

```

python3 src_jax/build_stage1_stats.py \
  --config <stage1_config> \
  --input <train_imagefolder> \
  --output <stage1_stat.pt> \
  --set stage_1.params.normalization_stat_path=<bootstrap_identity_stat.pt>

```

Trong đó file bootstrap identity stat chỉ cần chứa mean = 0 và var = 1. Mục tiêu là để pass tính stat đầu tiên không bị chuẩn hóa theo dataset khác, nhưng vẫn thỏa contract load stat của backend NNX.

4.4. Block stage_2

Một số key quan trọng:

- target: chỉ dùng để suy ra backbone NNX tương ứng;
- ckpt: có thể là file PyTorch .pt hoặc thư mục Orbx, nhưng trên branch này nó phải tương thích với shape của SiTDH;
- input_size, patch_size, in_channels;
- hidden_size, depth, num_heads: với SiTDH đây là các cặp encoder/decoder của backbone DH, còn với SiT đây là các scalar của backbone single-tower;
- class_dropout_prob: tỉ lệ label dropout cho CFG; với notebook CelebA một lớp trên đường JAX thì giá trị này được đặt về 0.0;
- các toggle như use_rope, use_rmsnorm, use_swiglu, wo_shift, use_pos_embed.

Với branch này, SiTDH sẽ được ánh xạ sang backend lightning_ddt, còn SiT sẽ được ánh xạ sang lightning_dit. Adapter vẫn giữ interface_class=sit cho cả hai, nên objective trên đường JAX vẫn là SiT / flow matching.

4.5. Block guidance

Hai mode chính:

- `cfg`: adapter dùng chính model làm conditional và unconditional pass, với nhãn null mặc định bằng `num_classes`;
- `autoguidance`: adapter load thêm `guidance.guidance_model` làm guide network.

4.6. Block training

Một số key được ánh xạ trực tiếp:

```
training:
  global_seed: 0
  epochs: 1400
  global_batch_size: 1024
  ema_decay: 0.9995
  num_workers: 4
  prefetch_factor: 2
  random_flip: true
  log_every: 100
  ckpt_every: 5000
  sample_every: 10000
  base_lr: 0.0002
  final_lr: 0.00002
  beta: [0.9, 0.95]
  wd: 0.0
  schedule_type: linear
  log_rae_latent_stats: false
  log_activation_stats: false
```

Lưu ý:

- nếu config chỉ cho `epochs`, adapter sẽ suy ra `total_steps` từ `num_train_samples`;
- optimizer mặc định vẫn là AdamW;
- scheduler được chuẩn hóa về các mode đơn giản như `constant`, `linear`, `cosine`;
- nếu `num_workers > 0`, có thể tăng `prefetch_factor` để queue trước nhiều batch hơn ở host-side PyTorch loader của đường JAX mà không đụng tới backend checkout;
- `random_flip` điều khiển việc chèn `RandomHorizontalFlip()` ở raw-image transform của đường JAX trước khi ảnh đi qua Stage 1; các notebook CelebA public của branch này bật cờ này cho train;
- nếu bật `log_rae_latent_stats=true`, đường JAX sẽ log `train_rae_latent_rms` và `train_rae_latent_var` cho latent Stage 1 thực sự đi vào Stage 2; tên metric được giữ nguyên để tương thích ngược kể cả khi Stage 1 là StabilityVAE;
- nếu bật `log_activation_stats=true`, đường JAX sẽ yêu cầu backend trả intermediate feature rồi log RMS/variance cho output của Stage 2 và từng block; DH sẽ log theo tên `train_sitdh_*`, còn single-tower sẽ log theo tên `train_sit_*`.

4.7. Block eval

Đường JAX giờ dùng block `eval` cho cả validation loss lẫn online FID. Các key `data_path`, `eval_every`, `batch_size`, `num_workers`, `prefetch_factor`, `random_flip`, `max_batches`, và `eval_model` sẽ điều khiển held-out validation loop trong `src_jax/train.py`. Với `fid_ref`, nên ưu tiên file `.pkl/.pickle`

được build bởi `src_jax/build_fid_stats.py`; nếu đưa file `.npz` cũ vào, adapter vẫn tự chuyển nó sang định dạng pickle mà backend NNX hiểu được. Trên đường JAX, metric FID-4K (`cfg=...`) mặc định chấm EMA; nếu bật `fid_eval_model`, adapter sẽ log thêm FID-4K/model (`cfg=...`) cho online model mà không thay metric EMA mặc định.

4.8. Override từ CLI

Các entrypoint JAX cho phép override nhanh bằng:

```
python3 src_jax/train.py \
  --config <config> \
  --set training.global_batch_size=256 \
  --set training.num_workers=16 \
  --set training.prefetch_factor=4 \
  --set eval.num_workers=16 \
  --set eval.prefetch_factor=4 \
  --set training.log_rae_latent_stats=true \
  --set training.log_activation_stats=true \
  --set guidance.scale=1.5
```

Mục tiêu là giữ đúng một schema YAML, nhưng vẫn cho phép chỉnh nhanh như CLI.

5. Vận hành, artifact và FID

5.1. Sampling Stage 2 trên đường JAX

5.1.1. Sampling nhanh

```
python3 src_jax/sample.py \
  --config <sample_config> \
  --seed 42 \
  --class-labels 207,360 \
  --output sample_jax.png
```

5.1.2. Sampling phân tán

```
python3 src_jax/sample_ddp.py \
  --config <sample_config> \
  --sample-dir samples_jax \
  --num-samples 50000 \
  --per-proc-batch-size 4 \
  --label-sampling equal \
  --fid-ref /path/to/reference_stats.npz
```

Artifact đầu ra:

- các file PNG theo index;
- các shard `.npz` theo rank nếu bật `-save-npz`;
- giá trị FID in ra terminal nếu cung cấp `-fid-ref`.

5.2. Build reference statistics cho FID

```
python src/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.npz \
  --device cpu \
  --batch-size 64 \
  --num-workers 32 \
  --num-threads 96
```

Đây là đường torch-fidelity chung cho nhánh XLA/PyTorch. Nếu cần fid_ref cho online FID ở JAX, nên build bằng detector backend-native:

```
python3 src_jax/build_fid_stats.py \
  --input /path/to/reference_images \
  --output /path/to/reference_stats.pkl \
  --batch-size 64 \
  --num-workers 8
```

5.3. Artifact của Stage 2 training

5.3.1. Đường XLA

Một experiment trong results/ thường có:

- log.txt,
- checkpoints/<step>.pt,
- fid_eval/step_<step>/... nếu bật online FID.

5.3.2. Đường JAX

Một run trong results_jax/ thường có:

- jax_adapter_config.json,
- thư mục checkpoint Orbx do backend quản lý,
- media và scalar trên wandb nếu bật -wandb,
- toàn bộ workdir có thể được đẩy lên Hugging Face nếu truyền -hf-repo-id.

5.4. Đẩy model lên Hugging Face

```
python3 src_jax/push_hf.py \
  --path results_jax/<run_name> \
  --repo-id <hf_user_or_org>/<repo_name>
```

Hoặc gắn thẳng vào lệnh train:

```
python3 src_jax/train.py \
  --config <config> \
  --data-path <train_root> \
  --hf-repo-id <hf_user_or_org>/<repo_name>
```

5.5. Các lưu ý vận hành quan trọng

- Đường JAX cố ý giữ nguyên schema YAML của repo để giảm chi phí bảo trì.
- `stage_2.ckpt` trên đường JAX có thể là PyTorch hoặc Orbx, nhưng cách load sẽ khác nhau.
- Đường JAX giờ cũng có validation loss định kỳ qua block `eval`, ngoài online FID.
- FID trên đường JAX nên dùng reference stats build bởi `src_jax/build_fid_stats.py` để cùng detector với backend; nếu cần, adapter vẫn chuyển `.npz` cũ sang pickle nội bộ.
- Online FID ở JAX mặc định chắt EMA. Nếu cần đối chiếu với model hiện thời, bật `eval.fid_eval_model=true` để log thêm metric FID-4K/model (`cfg=...`) bên cạnh series EMA mặc định.
- Nếu chỉ cần inference Stage 1 ở JAX, dùng `src_jax/stage1_sample.py`.
- Nếu cần stat Stage 1 hoặc reconstruction cả thư mục ở JAX, dùng `src_jax/build_stage1_stats.py` và `src_jax/reconstruct_folder.py`. Các lệnh này có thể dùng YAML chỉ có `stage_1`, không cần `stage_2.target`.
- Nếu chạy trên Kaggle với CelebA, bắt đầu từ notebook TPU `vaes-jax-celeba-kaggle-tpuv5e8-sitb.ipynb`. Bản này dựng `celeba256_imgfolder` từ dataset Kaggle giống branch `jax-sit-dh`, nhưng đổi Stage 1 sang `stage1.StabilityVAE`, đổi Stage 2 sang `single-tower SiT-B`, và bỏ hẳn bước bootstrap identity stat cùng pass `build_stage1_stats.py` bắt buộc.
- Nếu phần cứng là Kaggle TPU `v5e-8` và cần flow VAE đó trên TPU, dùng `vaes-jax-celeba-kaggle-tpuv5e8-sitb`. Notebook này giữ toàn bộ fix Kaggle/JAX hiện có, bật `training.random_flip=true`, viết config Stage 2 thành `CelebA256_SiT-B_StabilityVAE_jax-tpuv5e8.yaml`, bật luôn `training.log_rae_latent_stats` và `training.log_activation_stats=true`, mặc định `num_workers=16` và `prefetch_factor=4` cho cả train/eval, rồi dùng shape `single-tower hidden_size=768, depth=12, num_heads=12`.
- Nếu đã có run Orbx của flow `StabilityVAE + SiT-B` và cần resume, dùng notebook `vaes-jax-celeba-kaggle-tpuv5e8-sitb`. Notebook này dùng cùng pattern `resume-only` của bản DH, nhưng tự tìm thư mục `CelebA256_SiT-B_StabilityVAE` mới nhất và ép lại cả cấu hình `num_workers=16, prefetch_factor=4`, và hai cờ log diagnostics khi `resume`.
- Nếu cần huấn luyện Stage 1 kiểu GAN đầy đủ, hiện vẫn phải xem đây là phần chưa được JAX hóa trong branch này.

5.6. Checklist thực tế trước khi chạy dài

1. Xác nhận `stage_1` và `stage_2` khớp latent shape.
2. Xác nhận dữ liệu train và val đúng dạng `ImageFolder`.
3. Với flow RAE và dataset mới, build lại `stage1_stat.pt` thay vì tái dùng stat của ImageNet nếu muốn latent normalization khớp dữ liệu hơn.
4. Build `fid_ref` trước nếu muốn FID ổn định.
5. Export đủ biến wandb trước khi bật `-wandb`.
6. Nếu muốn upload lên Hugging Face, xác nhận token nằm trong biến môi trường được chỉ định bởi `-hf-token-env`.
7. Với JAX sampling số lượng lớn, chọn `-per-proc-batch-size` đủ an toàn theo bộ nhớ thiết bị.